



Habit Flow

Aplikasi Habit Tracker untuk membangun kebiasaan baik

Team and Responsibility



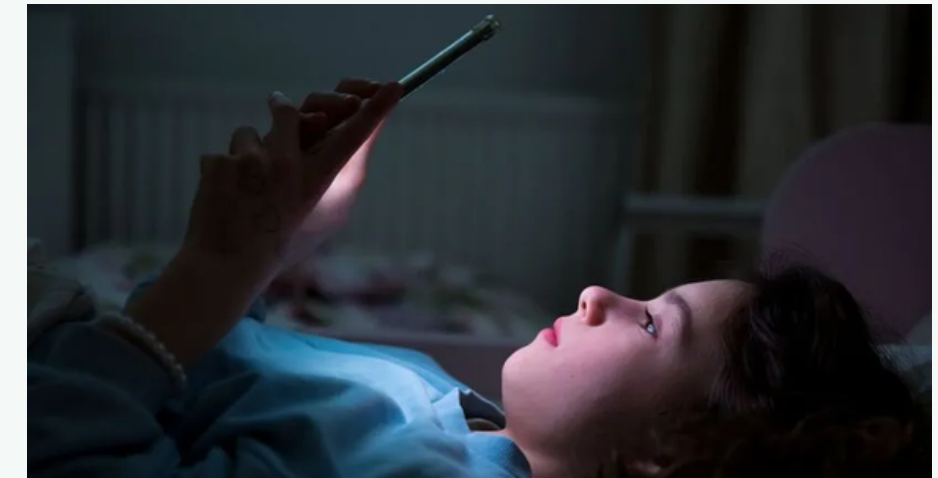
Anggota aKelompok:

- *Jecsen [2802438060]*
- *Perry Hardian [2802418802]*
- *Calvin Octo Fernando [2802401335]*
- *Marcelinus Wijaya Oey [2802414426]*

Latar Belakang

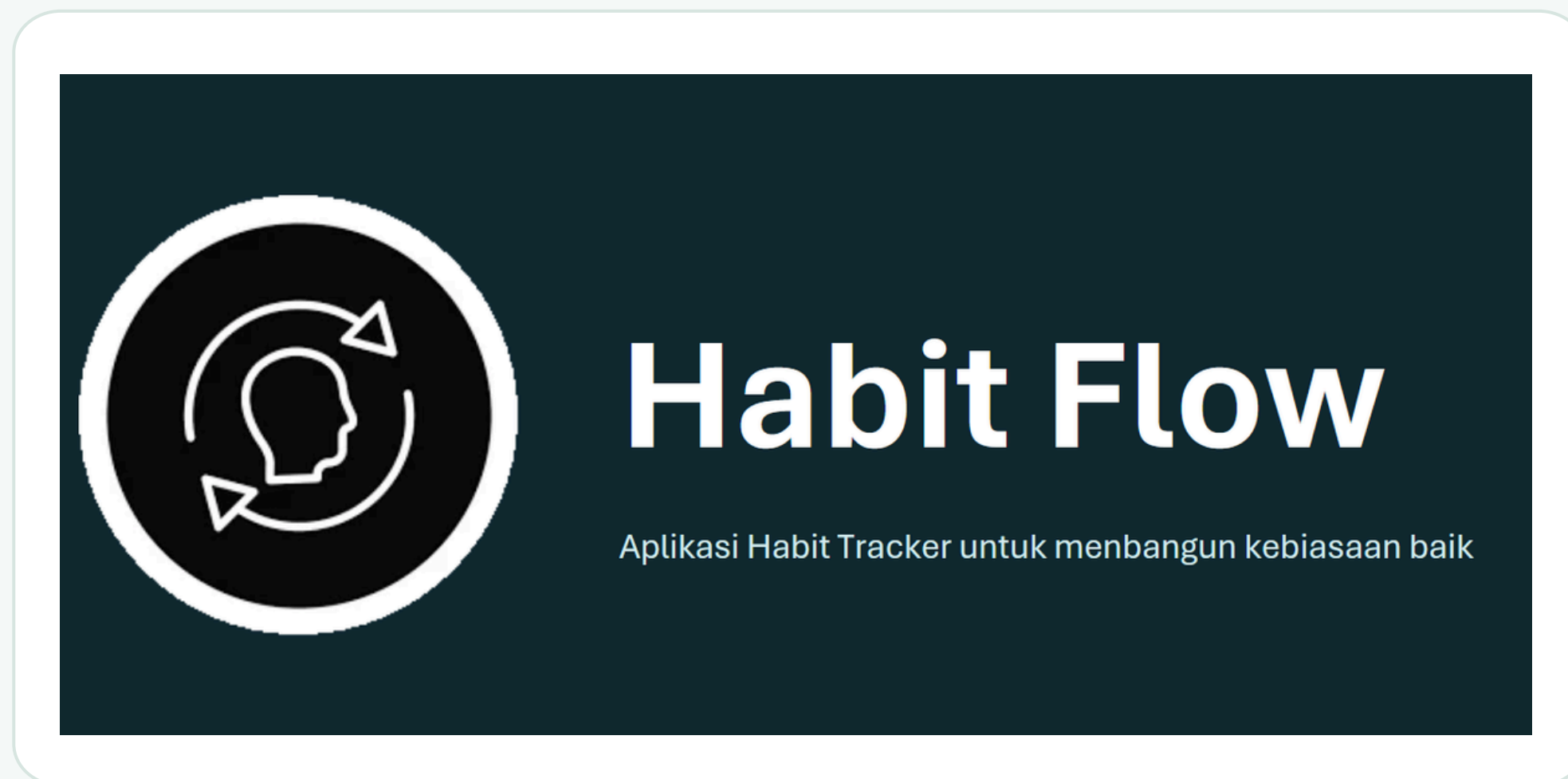
Ramai orang di jaman sekarang yang tidak bisa lepas dari kebiasaan buruk, seperti *doom scrolling*, *vape/rokok*, dll.

Meski banyak yang berusaha menghilangkan kebiasaan buruk dengan menggantinya dengan kebiasaan baik, lebih dari 90% percobaan berakhir gagal.



Solusi

Sehingga kami mendapat ide aplikasi habit tracker ini untuk membantu orang-orang yang ingin menghilangkan kebiasaan buruk maupun membangun kebiasaan hidup yang baik.



Tech Stack

Backend

NestJS, Node-js



Database

PostgreSQL



Validation

Zod



Testing

Jest, ts-jest



Frontend

Flutter, HTML, CSS, JS



DevOps

Docker



Version Control

Git dan GitHub



Design Pattern yang digunakan

Factory Method

Factory Method digunakan untuk membuat object konkret berdasarkan konfigurasi runtime.

Repository Pattern

Repository Pattern digunakan untuk memisahkan business logic dari detail penyimpanan data.

Adapter Pattern

Adapter pattern digunakan untuk membuat 2 atau lebih interface yang non compatible untuk bekerja sama.

Static Factory Pattern

Static Factory Method digunakan pada entity domain agar object hanya dibuat melalui method yang menjalankan validasi

Strategy Pattern

Strategy Pattern digunakan karena aplikasi memiliki beberapa strategi persistence yang dapat dipertukarkan di interface sama.

Service Layer pattern

Service Layer digunakan untuk mengatur perilaku use case aplikasi.

Software Architecture

Architecture yang digunakan: Microservice Architecture

Domain

Entity dan repository contract
src/domain

Application

Business rules dan use case
src/application/services

Presentation

NestJS controller, filter, validator
src/presentation

Infrastructure

PostgreSQL dan memory
repository
src/infrastructure

Dependency direction: Presentation -> Application -> Domain.

Outer layer Detail : NestJS dan PostgreSQL

Microservice yang diterapkan: API boundary, bounded capability, konfigurasi mandiri, repository abstraction.

System Flow

1. User membuka aplikasi dari browser

2. Frontend mengirim request ke NestJSREST API

3. NestJS Controller menerima request dan menjalankan validasi Zod.

4. Application Service menjalankan business rules habit.

5. Service mengakses data melalui Repository Interface.

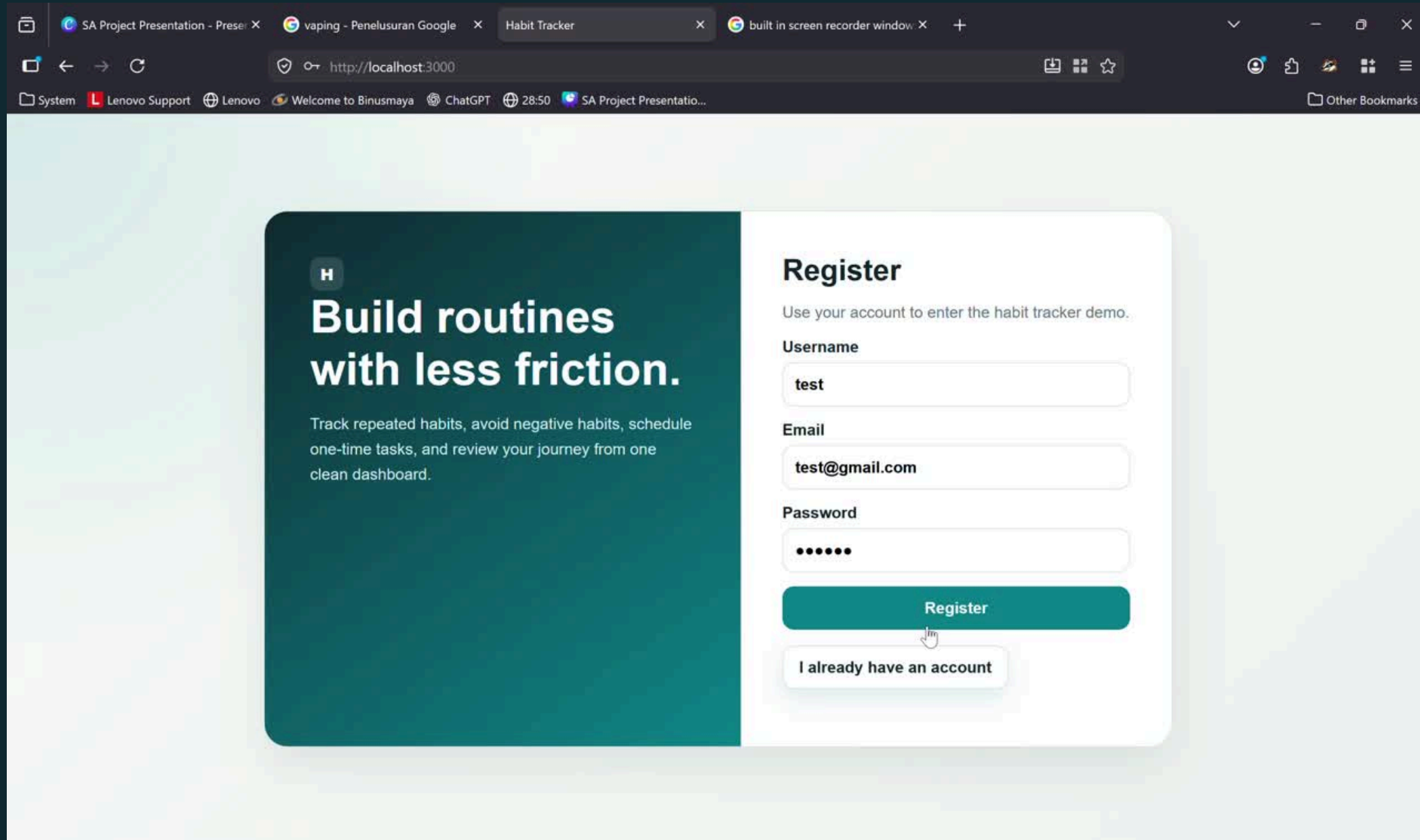
6. Repository menyimpan atau membaca data dari PostgreSQL / memory storage.

7. API mengembalikan JSON response ke frontend.

8. Frontend memperbarui tampilan Calendar, Journey, History, atau Profile.

Frontend -> NestJS Controller -> Validation -> Service Layer -> Repository -> Database -> Response -> UI

App Demo



GitHub: <https://github.com/Jecsen-uni/HabitFlow>

Terima Kasih